คู่มือการเรียก API ของหน้าเว็บ Board Game Lending System

เอกสารนี้อธิบายว่าเมื่อผู้ใช้กดปุ่มในแต่ละหน้าของเว็บ โค้ดส่วนไหนทำงาน เรียก endpoint ใด และผลลัพธ์ไปต่อที่ไหน ครอบคลุมทั้ง 3 หน้า (บอร์ดเกม, สมาชิก, ยืม-คืน)

---

1. ภาพรวมการทำงาน

- **ฐาน URL ของ API:** `http://localhost:3000/api/v1` (กำหนดไว้ที่ตัวแปร `API_BASE` ในไฟล์ `app.js`)
- **รูปแบบผลลัพธ์สำเร็จ:** `{ "message": "...", "data": ... }`
- **รูปแบบ error:** `{ "error": { "code": "...", "message": "..." } }`

---

2. ความหมายของคำว่า "ฝั่ง" ในคู่มือนี้

ฝั่ง	ไฟล์ที่เกี่ยวข้อง	ทำงานที่ไหน	หน้าที่
Frontend	`frontend/index.html`, `frontend/app.js`, `frontend/styles.css`	เบราว์เซอร์ของผู้ใช้	แสดงหน้าจอ รับการกดปุ่ม ส่ง request ไปที่ API แล้ววาดตารางใหม่
API	`src/index.js`, `src/validators.js`	Node.js (Express)	รับ request ตรวจข้อมูล สั่งฐานข้อมูล ส่ง response กลับ
DB	`src/db.js` + MySQL	Docker	เก็บข้อมูลจริง (`boardgames`, `members`, `borrowings`)
External	`src/services/external-api.js`	Node.js เรียกออกไปยัง ntfy.sh	ส่งการแจ้งเตือนเมื่อมีการยืมหรือคืน

> ส่วนที่เป็น Frontend จะเรียก API เท่านั้น ไม่เข้าถึงฐานข้อมูลโดยตรง

3. ขั้นตอนเมื่อเปิดหน้าเว็บ (ทำครั้งแรกอัตโนมัติ)

ฟังก์ชัน `init()` ใน `app.js` ทำงานทันทีที่โหลดหน้า โดยเรียก `loadAll()` ซึ่งยิง request ดังนี้

ลำดับ	Method	Endpoint	ทำอะไร	ฝั่งที่ทำงาน
1	GET	`/api/v1/boardgames`	ดึงบอร์ดเกมทั้งหมดไว้ใน `state.allGames`	Frontend → API → DB
2	GET	`/api/v1/members`	ดึงสมาชิกทั้งหมด ใช้วาดตารางสมาชิกและดรอปดาวน์	Frontend → API → DB
3	GET	`/api/v1/borrowings?status=BORROWED`	ดึงรายการที่กำลังยืม เพื่อรู้ว่าเกมไหนถูกยืมอยู่ (ใช้แสดงป้าย "ถูกยืมอยู่")	Frontend → API → DB
4	GET	`/api/v1/boardgames`	วาดตารางบอร์ดเกม (ซ้ำกับข้อ 1 ดูข้อสังเกตในหัวข้อ 9)	Frontend → API → DB
5	GET	`/api/v1/borrowings`	วาดตารางรายการยืม-คืน	Frontend → API → DB

หลังจากนั้น `app.js` จะวาดตารางสมาชิก เติมดรอปดาวน์ในฟอร์มยืม และแสดงตารางบอร์ดเกมกับตารางยืม-คืน

4. หน้าที่ 1: บอร์ดเกม (แท็บ "บอร์ดเกม")

4.1 ตารางสรุปปุ่มทั้งหมดในหน้า

ปุ่ม / การกระทำ	ฟังก์ชันใน `app.js`	Method	Endpoint	ฝั่งที่ทำงาน
ค้นหา	submit ของ `#search-form` → `loadGameTable()`	GET	`/api/v1/boardgames?players=N`	Frontend → API → DB
แสดงทั้งหมด	click ของ `#search-reset` → `loadGameTable()`	GET	`/api/v1/boardgames`	Frontend → API → DB
บันทึก (เพิ่มเกมใหม่)	submit ของ `#game-form` (ไม่มี id)	POST	`/api/v1/boardgames`	Frontend → API → DB
แก้ไข	`startEditGame()`	ไม่มี	ไม่เรียก API (เติมค่าในฟอร์มเท่านั้น)	Frontend
บันทึก (ตอนแก้ไข)	submit ของ `#game-form` (มี id)	PUT	`/api/v1/boardgames/:id`	Frontend → API → DB
ยกเลิกการแก้ไข	`resetGameForm()`	ไม่มี	ไม่เรียก API	Frontend
ลบ	`deleteGame()`	DELETE	`/api/v1/boardgames/:id`	Frontend → API → DB

4.2 รายละเอียดแต่ละการกระทำ

(ก) ค้นหาตามจำนวนผู้เล่น

1. ผู้ใช้กรอกจำนวนผู้เล่นแล้วกดปุ่ม "ค้นหา"
2. `app.js` ตรวจว่าเป็นจำนวนเต็มตั้งแต่ 1 ขึ้นไป (ตรวจที่ฝั่ง Frontend ก่อน) ถ้าไม่ผ่านจะแสดงข้อความใต้ช่อง
3. เก็บค่าไว้ที่ `state.searchPlayers` แล้วเรียก `loadGameTable()`
4. Frontend ส่ง `GET /api/v1/boardgames?players=4`
5. API (`src/index.js`) ตรวจว่า `players` เป็นจำนวนเต็มบวก แล้วกรองด้วยเงื่อนไข `min_players <= N AND max_players >= N`
6. ผลลัพธ์ถูกวาดในตาราง ถ้าไม่พบให้แสดงข้อความ "ไม่พบบอร์ดเกมที่รองรับ N คน"

(ข) แสดงทั้งหมด

1. ล้าง `state.searchPlayers` และช่องค้นหา

2. เรียก `loadGameTable()` ซึ่งส่ง `GET /api/v1/boardgames` (ไม่มี query)

> หมายเหตุ: เมื่ออยู่ในโหมดค้นหา การกดเพิ่ม แก้ไข หรือลบ เกม จะเรียก `loadAll()` ที่ยังคงใช้ค่า `state.searchPlayers` เดิม ตารางจึงยังคงแสดงผลที่กรองอยู่

(ค) เพิ่มบอร์ดเกมใหม่

1. ผู้ใช้กรอกชื่อ ผู้เล่นขั้นต่ำ และผู้เล่นขั้นสูง แล้วกด "บันทึก" (ตอนนั้นช่อง id ที่ซ่อนอยู่ว่าง)
2. `validateGameForm()` ตรวจที่ Frontend: ชื่อไม่ว่างและไม่เกิน 150 ตัวอักษร จำนวนผู้เล่นเป็นจำนวนเต็ม ≥ 1 และขั้นสูง ≥ ขั้นต่ำ
3. Frontend ส่ง `POST /api/v1/boardgames` พร้อม body `{ name, minPlayers, maxPlayers }`
4. API ตรวจด้วย `validateBoardgame()` แล้วบันทึกลงตาราง `boardgames`
   - ถ้าชื่อซ้ำ ฐานข้อมูลจะแจ้ง `ER_DUP_ENTRY` และ API ตอบ **409** `DUPLICATE_NAME`
   - ถ้าสำเร็จ ตอบ **201**
5. Frontend แสดงสถานะสำเร็จ รีเซ็ตฟอร์ม แล้วเรียก `loadAll()` เพื่อโหลดข้อมูลใหม่ทั้งหมด (ดูหัวข้อ 3)

(ง) แก้ไขบอร์ดเกม

1. กดปุ่ม "แก้ไข" ในแถวของเกม → `startEditGame()` เติมค่าเดิมลงในฟอร์ม ใส่ id ในช่องซ่อน และเปลี่ยนหัวข้อฟอร์มเป็น "แก้ไขบอร์ดเกม: ชื่อเกม" (ยังไม่มีการเรียก API)
2. ผู้ใช้แก้ค่าแล้วกด "บันทึก"
3. Frontend ส่ง `PUT /api/v1/boardgames/:id` พร้อม body เดิมที่มี `name`, `minPlayers`, `maxPlayers`
4. API ตรวจข้อมูลและ `UPDATE boardgames ...` ถ้าไม่พบ id ตอบ **404**, ถ้าชื่อซ้ำกับเกมอื่นตอบ **409**, ถ้าสำเร็จตอบ **200**
5. Frontend รีเซ็ตฟอร์มกลับเป็นโหมดเพิ่ม แล้วเรียก `loadAll()`

- ปุ่ม "ยกเลิกการแก้ไข" จะรีเซ็ตฟอร์มเท่านั้น ไม่มีการเรียก API

(จ) ลบบอร์ดเกม

1. กดปุ่ม "ลบ" → `deleteGame()` แสดงกล่องยืนยัน `confirm()`
2. ถ้าผู้ใช้ยืนยัน Frontend ส่ง `DELETE /api/v1/boardgames/:id`
3. API ลบแถวใน `boardgames` ถ้าไม่พบ id ตอบ **404** ถ้าสำเร็จตอบ **200**
4. ฐานข้อมูลลบประวัติ `borrowings` ของเกมนั้นด้วยอัตโนมัติ (`ON DELETE CASCADE`)
5. ถ้าฟอร์มกำลังแก้ไขเกมที่ถูกลบ ฟอร์มจะถูกรีเซ็ต แล้วเรียก `loadAll()`

---

5. หน้าที่ 2: สมาชิก (แท็บ "สมาชิก")

5.1 ตารางสรุปปุ่มทั้งหมดในหน้า

ปุ่ม / การกระทำ	ฟังก์ชันใน `app.js`	Method	Endpoint	ฝั่งที่ทำงาน
เพิ่มสมาชิก	submit ของ `#member-form`	POST	`/api/v1/members`	Frontend → API → DB
ประวัติการยืม	`showHistory(member)`	GET	`/api/v1/members/:id/borrowings`	Frontend → API → DB
ปิด (ประวัติ)	click ของ `#history-close`	ไม่มี	ไม่เรียก API (ซ่อนการ์ด)	Frontend
ลบ	`deleteMember()`	DELETE	`/api/v1/members/:id`	Frontend → API → DB

5.2 รายละเอียดแต่ละการกระทำ

(ก) เพิ่มสมาชิกใหม่

1. ผู้ใช้กรอกชื่อ-นามสกุล อีเมล เบอร์โทร แล้วกด "เพิ่มสมาชิก"
2. Frontend ตรวจทุกช่อง: ชื่อไม่ว่างและไม่เกิน 100 ตัวอักษร, อีเมลรูปแบบถูกต้องและไม่เกิน 100 ตัวอักษร, เบอร์โทรไม่ว่างและไม่เกิน 20 ตัวอักษร
3. Frontend ส่ง `POST /api/v1/members` พร้อม body `{ name, email, phone }`
4. API ตรวจด้วย `validateMember()` แล้วบันทึกลงตาราง `members`
   - ถ้าอีเมลซ้ำ ตอบ **409** `DUPLICATE_EMAIL` (Frontend จะแสดงข้อความใต้ช่องอีเมล)
   - ถ้าสำเร็จ ตอบ **201**
5. Frontend ล้างฟอร์ม แสดงสถานะสำเร็จ แล้วเรียก `loadAll()`

(ข) ดูประวัติการยืมของสมาชิก

1. กดปุ่ม "ประวัติการยืม" ในแถวของสมาชิก → `showHistory()` เปิดการ์ด "ประวัติการยืมของ ชื่อ" และแสดงข้อความกำลังโหลด
2. Frontend ส่ง `GET /api/v1/members/:id/borrowings`
3. API ตรวจว่ามีสมาชิกคนนี้จริง (ไม่พบ → **404**) แล้วดึงประวัติจาก `borrowings` JOIN `boardgames` และ `members` เรียงจากล่าสุด
4. Frontend วาดตารางประวัติ (ชื่อเกม วันที่ยืม วันที่คืน สถานะ) ถ้าไม่มีประวัติจะแสดงข้อความว่าสมาชิกยังไม่เคยยืม

- การกด "ปิด" เพียงซ่อนการ์ด ไม่มีการเรียก API

(ค) ลบสมาชิก

1. กดปุ่ม "ลบ" → `deleteMember()` แสดงกล่องยืนยัน
2. ถ้าผู้ใช้ยืนยัน Frontend ส่ง `DELETE /api/v1/members/:id`
3. API ลบแถวใน `members` ถ้าไม่พบ id ตอบ **404** ถ้าสำเร็จตอบ **200**
4. ประวัติการยืมของสมาชิกคนนั้นถูกลบอัตโนมัติ (`ON DELETE CASCADE`)
5. Frontend ซ่อนการ์ดประวัติ แล้วเรียก `loadAll()`

---

6. หน้าที่ 3: ยืม-คืน (แท็บ "ยืม-คืน")

6.1 ตารางสรุปปุ่มและตัวกรองทั้งหมดในหน้า

ปุ่ม / การกระทำ	ฟังก์ชันใน `app.js`	Method	Endpoint	ฝั่งที่ทำงาน
บันทึกการยืม	submit ของ `#borrow-form`	POST	`/api/v1/borrowings`	Frontend → API → DB → External
เปลี่ยนวันที่ "ยืมตั้งแต่"	change ของ `#borrow-from` → `loadBorrowings()`	GET	`/api/v1/borrowings?from=...`	Frontend → API → DB
เปลี่ยนวันที่ "ยืมถึง"	change ของ `#borrow-to` → `loadBorrowings()`	GET	`/api/v1/borrowings?to=...`	Frontend → API → DB
เปลี่ยนตัวกรอง "สถานะ"	ไม่มี listener (ดูข้อสังเกตในหัวข้อ 9)	-	-	-
คืนบอร์ดเกม	`returnGame()`	PATCH	`/api/v1/borrowings/:id/return`	Frontend → API → DB → External

6.2 รายละเอียดแต่ละการกระทำ

(ก) บันทึกการยืม

1. ผู้ใช้เลือกสมาชิก เลือกบอร์ดเกม (เลือกได้ทุกเกม แต่เกมที่กำลังถูกยืมจะมีคำว่า "(ถูกยืมอยู่)" ต่อท้าย) และกรอกวันที่ยืม (ถ้าไม่กรอกจะเป็นวันนี้)

2. Frontend ตรวจว่าเลือกสมาชิกและบอร์ดเกมแล้ว แล้วสร้าง payload `{ memberId, boardgameId }` และเพิ่ม `borrowDate` ถ้ามีการกรอก
3. Frontend ส่ง `POST /api/v1/borrowings`
4. API ทำงานใน **transaction** ดังนี้
    1. ตรวจ `memberId`, `boardgameId` เป็นจำนวนเต็มบวก และ `borrowDate` เป็น `YYYY-MM-DD` (ถ้ามี) — ไม่ผ่านตอบ **400**
    2. เปิด transaction แล้ว `SELECT ... FOR UPDATE` ล็อกแถวของบอร์ดเกม เพื่อกันการยืมพร้อมกันสองคน — ไม่พบเกมตอบ **404**
    3. ตรวจว่ามีสมาชิกคนนั้น — ไม่พบตอบ **404**
    4. ตรวจว่าเกมนี้มีรายการ `status = 'BORROWED'` อยู่หรือไม่ — มีแล้วตอบ **409** `GAME_NOT_AVAILABLE`
    5. `INSERT` ลงตาราง `borrowings` โดยสถานะเริ่มต้นเป็น `BORROWED` แล้ว `COMMIT`
5. หลัง commit API เรียก `notifyBorrowed()` ไปยัง ntfy.sh แบบไม่รอผลลัพธ์ (ถ้าแจ้งเตือนล้มเหลว รายการยืมยังคงอยู่)
6. API ตอบ **201** พร้อม `{ id, memberId, boardgameId, borrowDate, returnDate: null, status: "BORROWED" }`
7. Frontend ล้างฟอร์ม แสดงสถานะสำเร็จ แล้วเรียก `loadAll()`
    - ถ้าได้ 409 Frontend จะแสดงข้อความ "บอร์ดเกมนี้ถูกยืมอยู่ในขณะนี้" ใต้ช่องบอร์ดเกม

(ข) ตัวกรองรายการยืม-คืน (วันที่)

1. ผู้ใช้เปลี่ยนวันที่ "ยืมตั้งแต่" หรือ "ยืมถึง" → event `change` เรียก `loadBorrowings()`
2. `loadBorrowings()` อ่านค่าตัวกรองทั้งหมดจากหน้าจอ ประกอบ query string แล้วส่ง `GET /api/v1/borrowings?status=...&from=...&to=...` (เฉพาะตัวที่มีค่า)
3. API ตรวจแต่ละตัวกรอง (status ต้องเป็น BORROWED หรือ RETURNED, วันที่ต้องเป็น YYYY-MM-DD) แล้วสร้าง SQL `WHERE` ตามเงื่อนไขที่ส่งมา
4. Frontend วาดตารางใหม่ ถ้าไม่พบรายการให้แสดงข้อความ "ไม่มีรายการยืม-คืนตามเงื่อนไขที่เลือก"

(ค) คืนบอร์ดเกม

1. ปุ่ม "คืนบอร์ดเกม" จะปรากฏเฉพาะแถวที่สถานะเป็น "กำลังยืม"
2. กดปุ่ม → `returnGame()` ส่ง `PATCH /api/v1/borrowings/:id/return` พร้อม body `{}` (Frontend ไม่ได้ส่ง `returnDate`)
3. API ทำดังนี้
    1. ค้นหารายการยืม — ไม่พบตอบ **404**
    2. ถ้าสถานะเป็น RETURNED อยู่แล้ว ตอบ **409** `ALREADY_RETURNED`
    3. ถ้าไม่ได้ส่ง `returnDate` ใช้วันนี้เป็นค่าเริ่มต้น
    4. ตรวจว่า `returnDate` ไม่ก่อน `borrowDate` — ไม่ผ่านตอบ **400**
    5. `UPDATE` ตั้ง `return_date` และ `status = 'RETURNED'` โดยมีเงื่อนไข `status = 'BORROWED'` ใน WHERE เพื่อกันการคืนซ้ำพร้อมกัน

6. ดึงรายการที่อัปเดตแล้วพร้อมชื่อสมาชิกและชื่อเกม

4. API เรียก `notifyReturned()` ไปยัง ntfy.sh แบบไม่รอผลลัพธ์ แล้วตอบ **200** พร้อมข้อมูลรายการ
5. Frontend แสดงข้อความ "คืนบอร์ดเกมสำเร็จ" แล้วเรียก `loadAll()`

---

7. Endpoint ที่หน้าเว็บไม่ได้เรียก (มีใน API แต่ไม่มีปุ่มในหน้าเว็บ)

Method	Endpoint	ใช้ทำอะไร	วิธีเรียก
GET	`/`	ตรวจว่า server ทำงานอยู่	เปิดเบราว์เซอร์ที่ `http://localhost:3000/`
GET	`/api/v1/boardgames/:id`	ดึงบอร์ดเกมรายตัว	Postman / curl
GET	`/api/v1/members/:id`	ดึงสมาชิกรายคน	Postman / curl
PUT	`/api/v1/members/:id`	แก้ไขสมาชิก (หน้าเว็บไม่มีฟอร์มแก้ไขสมาชิก)	Postman / curl
GET	`/api/v1/borrowings/:id`	ดึงรายการยืมรายตัว	Postman / curl
GET	`/api/v1/external/notify?message=...`	ส่งข้อความแจ้งเตือนเอง (ใช้ทดสอบระบบ ntfy.sh)	เปิดเบราว์เซอร์หรือ Postman

เอกสารทุก endpoint ดูได้ที่ `http://localhost:3000/api-docs` (Swagger UI)

---

8. สรุปการเชื่อมโยง endpoint กับหน้าเว็บ

Endpoint	บอร์ดเกม	สมาชิก	ยืม-คืน
GET `/api/v1/boardgames`	✓ (โหลด, ค้นหา, แสดงทั้งหมด)		
GET `/api/v1/boardgames/:id`			
POST `/api/v1/boardgames`	✓		
PUT `/api/v1/boardgames/:id`	✓		
DELETE `/api/v1/boardgames/:id`	✓		
GET `/api/v1/members`		✓	
GET `/api/v1/members/:id`			
GET `/api/v1/members/:id/borrowings`		✓ (ประวัติการยืม)	
POST `/api/v1/members`		✓	
PUT `/api/v1/members/:id`			
DELETE `/api/v1/members/:id`		✓	
GET `/api/v1/borrowings`	✓ (ป้าย "ถูกยืมอยู่" ผ่าน `?status=BORROWED`)		✓
GET `/api/v1/borrowings/:id`			
POST `/api/v1/borrowings`			✓
PATCH `/api/v1/borrowings/:id/return`			✓
GET `/api/v1/external/notify`			

---

9. ข้อสังเกตจากการอ่านโค้ด

1. **ตัวกรองสถานะในหน้ายืม-คืนยังไม่ทำงานทันที:** `app.js` ผูก `change` กับช่องวันที่ทั้งสองช่อง แต่ไม่ได้ผูกกับ `#borrow-filter` (ตัวเลือกสถานะ) ผู้ใช้ต้องเปลี่ยนวันที่หรือทำรายการอื่นก่อน ตารางจึงจะอัปเดตตามสถานะ

2. **ตอนโหลดหน้า GET `/api/v1/boardgames` ถูกเรียกสองครั้ง:** ครั้งแรกอยู่ใน `loadAll()` และอีกครั้งใน `loadGameTable()` เป็นการเรียกซ้ำที่ไม่จำเป็น
3. **การทำรายการแต่ละครั้งจะโหลดข้อมูลใหม่ทั้งหมด:** ทุก POST/PUT/DELETE/PATCH จะเรียก `loadAll()` ซึ่งยิง GET ประมาณ 5 ครั้ง เป็นการทำงานที่ง่ายแต่ไม่ประหยัด
4. **การลบบอร์ดเกมหรือสมาชิกจะลบประวัติการยืมด้วย:** ฐานข้อมูลตั้ง `ON DELETE CASCADE` ไว้ และฝั่ง API ไม่ได้ป้องกันการลบเกมที่กำลังถูกยืม ควรระวังเมื่อใช้งานจริง
5. **การแจ้งเตือนไม่กระทบรายการที่บันทึกแล้ว:** การยืมและการคืนจะถูกบันทึกลงฐานข้อมูลก่อน แล้วจึงส่งแจ้งเตือนไปยัง ntfy.sh โดยไม่รอผล ถ้าส่งไม่สำเร็จ ระบบจะบันทึก log เท่านั้น
6. **ไม่มีฟอร์มแก้ไขสมาชิกในหน้าเว็บ** ดังนั้น PUT `/api/v1/members/:id` จึงไม่ถูกเรียกจากหน้าเว็บ แต่ยังใช้ได้ผ่าน Postman
7. **ค่า `API_BASE` เป็น `http://localhost:3000/api/v1`:** หากนำระบบไป deploy ต้องแก้ค่านี้ใน `app.js` และตั้งค่า `ALLOWED_ORIGIN` ในไฟล์ `.env` ให้ตรงกับโดเมนของหน้าเว็บ มิฉะนั้นเบราว์เซอร์จะบล็อก request (CORS)